How are we doing? Please help us improve Stack Overflow. Take our short survey

How does SSL really work?

Asked 13 years, 10 months ago Modified 5 years, 5 months ago Viewed 61k times

164

How does SSL work?

Where is the certificate installed on the client (or browser?) and the server (or web server?)?

How does the trust/encryption/authentication process start when you enter the URL into the browser and get the page from the server?

How does the HTTPS protocol recognize the certificate? Why can't HTTP work with certificates when it is the certificates which do all the trust/encryption/authentication work?

ssl ssl-certificate

Share Improve this question

Follow

edited Jun 20, 2014 at 0:04

Warren Dew

8,702 3 30 43

asked Jan 22, 2009 at 19:41

Vicky

11k 11 35 29

29 I think this is a reasonable question - understanding how SSL works is step 1, implementing it correctly is step 2 through step infinity. – synthesizerpatel Feb 22, 2012 at 12:35

6 See also: security.stackexchange.com/questions/20803/how-does-ssl-work – Luc Oct 5, 2012 at 19:07

6 Here's a good run-through of the https handshake process at a byte level – Rob Church Apr 29, 2013 at 13:55

7 @StingyJack Don't be a policy nazi here. People come looking for help. Don't deny them all assistance because you find the question does not perfectly match with the rules. – Koray Tugay Oct 7, 2013 at 18:13

1 @KorayTugay - noone is denying assistance. This does belong on Security or Sysadmin where it is better targeted, but OP would typically benefit in this forum by adding some bit of programming context instead of posting a general IT question. How many people get questions shut down when they are not tied to a specific programming problem? Probably too many, hence my nudging OP to make that association. – StingyJack Oct 8, 2013 at 17:57

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email Sign up with Google Sign up with GitHub Sign up with Facebook

167

Note: I wrote my original answer very hastily, but since then, this has turned into a fairly popular question/answer, so I have expanded it a bit and made it more precise.

TLS Capabilities

"SSL" is the name that is most often used to refer to this protocol, but SSL specifically refers to the proprietary protocol designed by Netscape in the mid 90's. "TLS" is an IETF standard that is based on SSL, so I will use TLS in my answer. These days, the odds are that nearly all of your secure connections on the web are really using TLS, not SSL.

TLS has several capabilities:

1. Encrypt your application layer data. (In your case, the application layer protocol is HTTP.)
2. Authenticate the server to the client.
3. Authenticate the client to the server.

#1 and #2 are very common. #3 is less common. You seem to be focusing on #2, so I'll explain that part.

Authentication

A server authenticates itself to a client using a certificate. A certificate is a blob of data[1] that contains information about a website:

- Domain name
- Public key
- The company that owns it
- When it was issued
- When it expires
- Who issued it
- Etc.

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email | Sign up with Google | Sign up with GitHub | Sign up with Facebook

But what if an adversary could modify packets sent to and from the server, and what if that adversary modified the certificate you were presented with and inserted their own public key or changed any other important details? If that happened, the adversary could intercept and modify any messages that you thought were securely encrypted.

To prevent this very attack, the certificate is cryptographically signed by somebody else's private key in such a way that the signature can be verified by anybody who has the corresponding public key. Let's call this key pair KP2, to make it clear that these are not the same keys that the server is using.

Certificate Authorities

So who created KP2? Who signed the certificate?

Oversimplifying a bit, a *certificate authority* creates KP2, and they sell the service of using their private key to sign certificates for other organizations. For example, I create a certificate and I pay a company like Verisign to sign it with their private key.[3] Since nobody but Verisign has access to this private key, none of us can forge this signature.

And how would I personally get ahold of the public key in KP2 in order to verify that signature?

Well we've already seen that a certificate can hold a public key — and computer scientists love recursion — so why not put the KP2 public key into a certificate and distribute it that way? This sounds a little crazy at first, but in fact that's exactly how it works. Continuing with the Verisign example, Verisign produces a certificate that includes information about who they are, what types of things they are allowed to sign (other certificates), and their public key.

Now if I have a copy of that Verisign certificate, I can use that to validate the signature on the server certificate for the website I want to visit. Easy, right?!

Well, not so fast. I had to get the Verisign certificate from *somewhere*. What if somebody spoofs the Verisign certificate and puts their own public key in there? Then they can forge the signature on the server's certificate, and we're right back where we started: a man-in-the-middle attack.

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email | Sign up with Google | Sign up with GitHub | Sign up with Facebook

Hopefully you can already see that this recursive approach is just turtles/certificates all the way down. Where does it stop?

Since we can't create an infinite number of certificates, the certificate chain obviously has to stop *somewhere*, and that's done by including a certificate in the chain that is *self-signed*.

> I'll pause for a moment while you pick up the pieces of brain matter from your head exploding. *Self-signed?!*

Yes, at the end of the certificate chain (a.k.a. the "root"), there will be a certificate that uses it's own keypair to sign itself. This eliminates the infinite recursion problem, but it doesn't fix the authentication problem. Anybody can create a self-signed certificate that says anything on it, just like I can create a fake Princeton diploma that says I triple majored in politics, theoretical physics, and applied butt-kicking and then sign my own name at the bottom.

The [somewhat unexciting] solution to this problem is just to pick some set of self-signed certificates that you explicitly trust. For example, I might say, "I *trust* this Verisign self-signed certificate."

With that explicit trust in place, now I can validate the entire certificate chain. No matter how many certificates there are in the chain, I can validate each signature all the way down to the root. When I get to the root, I can check whether that root certificate is one that I explicitly trust. If so, then I can trust the entire chain.

Conferred Trust

Authentication in TLS uses a system of *conferred trust*. If I want to hire an auto mechanic, I may not trust any random mechanic that I find. But maybe my friend vouches for a particular mechanic. Since I trust my friend, then I can trust that mechanic.

When you buy a computer or download a browser, it comes with a few hundred root certificates that it explicitly trusts.[4] The companies that own and operate those certificates can confer that trust to other organizations by signing their certificates.

This is far from a perfect system. Some times a CA may issue a certificate erroneously. In those cases, the certificate may need to be revoked. Revocation is tricky since the issued certificate will always be cryptographically correct; an out-of-

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email | Sign up with Google | Sign up with GitHub | Sign up with Facebook

Verisign and steal their root signing key, then you could spoof *any* certificate in the world. Notice that this doesn't just affect Verisign customers: even if my certificate is signed by Thawte (a competitor to Verisign), that doesn't matter. My certificate can still be forged using the compromised signing key from Verisign.

This isn't just theoretical. It has happened in the wild. DigiNotar was famously hacked and subsequently went bankrupt. Comodo was also hacked, but inexplicably they remain in business to this day.

Even when CAs aren't directly compromised, there are other threats in this system. For example, a government use legal coercion to compel a CA to sign a forged certificate. Your employer may install their own CA certificate on your employee computer. In these various cases, traffic that you expect to be "secure" is actually completely visible/modifiable to the organization that controls that certificate.

Some replacements have been suggested, including Convergence, TACK, and DANE.

Endnotes

[1] TLS certificate data is formatted according to the X.509 standard. X.509 is based on ASN.1 ("Abstract Syntax Notation #1"), which means that it is *not* a binary data format. Therefore, X.509 must be *encoded* to a binary format. DER and PEM are the two most common encodings that I know of.

[2] In practice, the protocol actually switches over to a symmetric cipher, but that's a detail that's not relevant to your question.

[3] Presumable, the CA actually validates who you are before signing your certificate. If they didn't do that, then I could just create a certificate for google.com and ask a CA to sign it. With that certificiate, I could man-in-the-middle any "secure" connection to google.com. Therefore, the validation step is a very important factor in the operation of a CA. Unfortunately, it's not very clear how rigorous this validation process is at the hundreds of CAs around the world.

[4] See Mozilla's list of trusted CAs.

Share Improve this answer Follow

edited Aug 9, 2014 at 18:11

answered Sep 11, 2011 at 22:26

Mark E. Haase
25.4k 11 64 72

What is a private key? – Koray Tugay Oct 4, 2013 at 21:52

2 @mamdouhalramadan "the public key is ... sent to the website to decrypt the data". How can

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email | Sign up with Google | Sign up with GitHub | Sign up with Facebook

60

HTTPS is combination of **HTTP** and **SSL(Secure Socket Layer)** to provide encrypted communication between client (browser) and web server (application is hosted here).

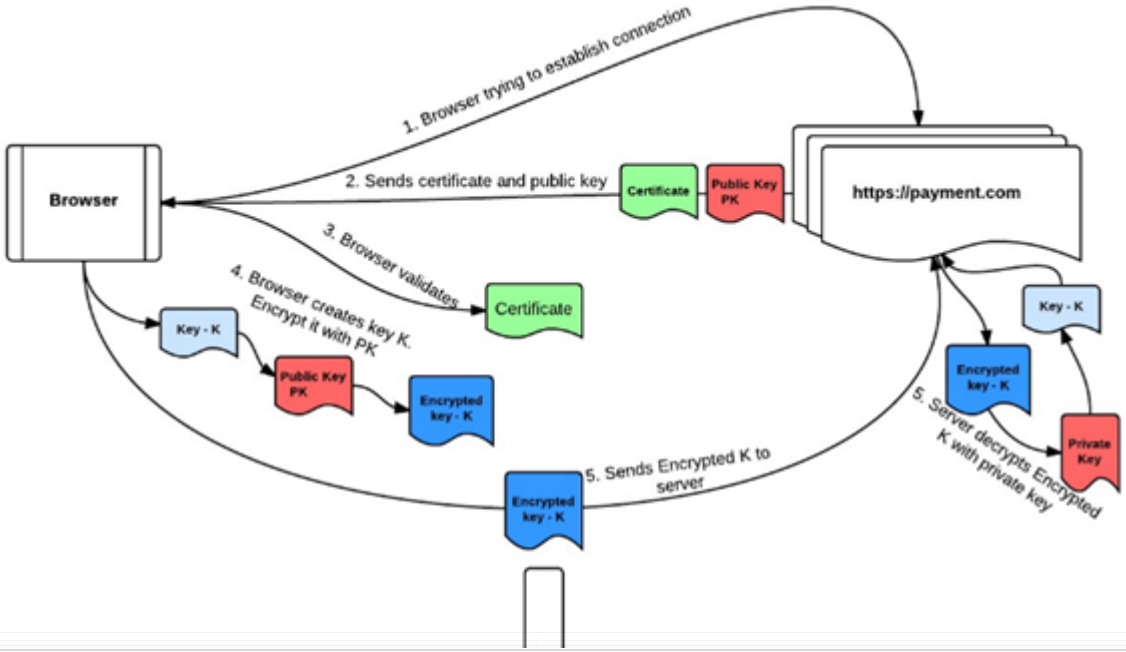
Why is it needed?

HTTPS encrypts data that is transmitted from browser to server over the network. So, no one can sniff the data during transmission.

How HTTPS connection is established between browser and web server?

1. Browser tries to connect to the https://payment.com.
2. payment.com server sends a certificate to the browser. This certificate includes payment.com server's public key, and some evidence that this public key actually belongs to payment.com.
3. Browser verifies the certificate to confirm that it has the proper public key for payment.com.
4. Browser chooses a random new symmetric key K to use for its connection to payment.com server. It encrypts K under payment.com public key.
5. payment.com decrypts K using its private key. Now both browser and the payment server know K, but no one else does.
6. Anytime browser wants to send something to payment.com, it encrypts it under K; the payment.com server decrypts it upon receipt. Anytime the payment.com server wants to send something to your browser, it encrypts it under K.

This flow can be represented by the following diagram:



Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email | Sign up with Google | Sign up with GitHub | Sign up with Facebook

Share Improve this answer Follow

edited Aug 5, 2014 at 18:29

answered Aug 5, 2014 at 3:41

sanjay_kv
709 5 4

3 The part about encrypting and sending the session key and decrypting it at the server is complete and utter rubbish. The session key is never transmitted at all: it is established via a secure key negoatiaon algorithm. Please check your facts before posting nonsense like this. RFC 2246. – user207421 Feb 7, 2017 at 21:43

Why browser dont use server's public key to encrypt data when post it to server, instead of creating a random new symmetric key K at step 4? – KevinBui Nov 29, 2017 at 6:33

1 @KevinBui Because sending the response from the server would require the client to have a key pair, and because asymmetric encryption is very slow. – user207421 Dec 14, 2019 at 13:53

Agree with @user207421 The answer is not correct Client only sends pre_master_secret key encrypted with the use of server's publickey. pre_master_secret is then used both by client and server to compute symmetric master_key (master_key = f(pre_master_secret, "master secret", ClientHello.random + ServerHello.random) master_key used to encrypt all the messages sent afterwards. – guybydefault Jan 11, 2021 at 18:23

4

I have written a small blog post which discusses the process briefly. Please feel free to take a look.

SSL Handshake

A small snippet from the same is as follows:

"Client makes a request to the server over HTTPS. Server sends a copy of its SSL certificate + public key. After verifying the identity of the server with its local trusted CA store, client generates a secret session key, encrypts it using the server's public key and sends it. Server decrypts the secret session key using its private key and sends an acknowledgment to the client. Secure channel established."

Share Improve this answer Follow

answered Jan 11, 2016 at 10:28

Abhishek Jain
4,358 7 32 49

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email | Sign up with Google | Sign up with GitHub | Sign up with Facebook

"After verifying the identity of the server with its local trusted CA store" - this is not strictly true. I can use a self-signed certificate and HTTPS would work - I can get a secure HTTPS connection to **a server**. The trusted certificate comes in only when I want to make sure that I am talking to the **right** server. – Teddy Jan 12, 2017 at 6:06

The part about encrypting and sending the session key and decrypting it at the server is complete and utter rubbish. The session key is never transmitted at all: it is established via a secure key negoatiaon algorithm. Please check your facts before posting nonsense like this. RFC 2246. – user207421 Feb 7, 2017 at 21:39

@Teddy That's not correct. Certificate trust checking is a require part of SSL. It can be bypassed but it is normally in effect: self-signed certificates don't work without special measures of one kind or another. – user207421 Feb 7, 2017 at 21:41

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email | Sign up with Google | Sign up with GitHub | Sign up with Facebook

3

Mehaase has explained it in details already. I will add my 2 cents to this series. I have many blogposts revolving around SSL handshake and certificates. While most of this revolves around IIS web server, the post is still relevant to SSL/TLS handshake in general. Here are few for your reference:

- SSL Handshake and IIS
- Client certificate Authentication in SSL Handshake

> Do not treat **CERTIFICATES** & **SSL** as one topic. Treat them as 2 different topics and then try to see who they work in conjunction. This will help you answer the question.

Establishing trust between communicating parties via Certificate Store

SSL/TLS communication works solely on the basis of trust. Every computer (client/server) on the internet has a list of Root CA's and Intermediate CA's that it maintains. These are periodically updated. During SSL handshake this is used as a reference to establish trust. For exampe, during SSL handshake, when the client provides a certificate to the server. The server will try to cehck whether the CA who issued the cert is present in its list of CA's . When it cannot do this, it declares that it was unable to do the certificate chain verification. (This is a part of the answer. It also looks at **AIA** for this.) The client also does a similar verification for the server certificate which it receives in Server Hello. On Windows, you can see the certificate stores for client & Server via PowerShell. Execute the below from a PowerShell console.

> PS Cert:> ls Location : CurrentUser StoreNames : {TrustedPublisher, ClientAuthIssuer, Root, UserDS...}
>
> Location : LocalMachine StoreNames : {TrustedPublisher, ClientAuthIssuer, Remote Desktop, Root...}

Browsers like Firefox and Opera don't rely on underlying OS for certificate management. They maintain their own separate certificate stores.

The SSL handshake uses both Symmetric & Public Key Cryptography. Server Authentication happens by default. Client Authentication is optional and depends if the Server endpoint is configured to authenticate the client or not. Refer my blog post as I have explained this in detail.

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email | Sign up with Google | Sign up with GitHub | Sign up with Facebook

trust/encryption/authentication work?

Certificates is simply a file whose format is defined by X.509 standard. It is a electronic document which proves the identity of a communicating party. **HTTPS = HTTP + SSL** is a protocol which defines the guidelines as to how 2 parties should communicate with each other.

MORE INFORMATION

- In order to understand certificates you will have to understand what certificates are and also read about Certificate Management. These is important.
- Once this is understood, then proceed with TLS/SSL handshake. You may refer the RFC's for this. But they are skeleton which define the guidelines. There are several blogposts including mine which explain this in detail.

If the above activity is done, then you will have a fair understanding of Certificates and SSL.

Share Improve this answer Follow

answered Jun 23, 2017 at 12:45

Kaushal Kumar Panday
2,279 11 22

Highly active question. Earn 10 reputation (not counting the association bonus) in order to answer this question. The reputation requirement helps protect this question from spam and non-answer activity.

Join Stack Overflow to find the best answer to your technical question, help others answer theirs.

Sign up with email | Sign up with Google | Sign up with GitHub | Sign up with Facebook